

100 Days of Linux - Day 064

Title: Accepting User Input with read

Today's Goal

Learn how to accept user input in Bash scripts using the **read** command.

Why This Matters

Interactive scripts ask users for names, file paths, project names, and other information. This is a key step toward writing practical automation scripts.

Command of the Day: read

Syntax:

```
read VARIABLE
```

Examples:

```
echo "Enter your name:"  
read NAME  
echo "Hello, $NAME"
```

Useful Options:

```
read -p "Enter your age: " AGE  
read -s -p "Enter password: " PASSWORD (hidden input)  
echo
```

Beginner Lesson

The **read** command pauses the script and waits for the user to type something. The entered value is stored in a variable and can be used later in the script.

Practice Questions

1. Create a script named `user_input.sh` that asks for your name and prints a welcome message.
2. Modify the script to ask for your age and display both your name and age.
3. Use the **-p** option to display the prompt inside the `read` command.
4. Create a script that asks for a project name and prints 'Project created: `PROJECT_NAME`'.
5. Run the script and then display your current working directory.

Hints

Use echo before read, or use read -p to display the prompt directly.

Mini Challenge

Create `student_registration.sh` that asks for a student's name, age, city, and favorite Linux command, then prints a formatted summary.

Common Mistakes

Forgetting to use the dollar sign (\$) when displaying the variable after using read.

Did You Know?

Many installation and deployment scripts use read to ask users for configuration values before continuing.

Pro Tip

Use meaningful variable names such as USER_NAME or PROJECT_NAME to make scripts easier to understand.

Answer Key (Instructor Only)

Q1: echo "Enter your name: "; read NAME; echo "Welcome \$NAME"

Q2: read AGE; echo "\$NAME is \$AGE years old"

Q3: read -p "Enter your age: " AGE

Q4: read -p "Project name: " PROJECT; echo "Project created: \$PROJECT"

Q5: bash user_input.sh && pwd

Homework

Create three interactive scripts that ask for different inputs and display personalized output.

Next Day Preview

Tomorrow you'll learn how to use command-line arguments in Bash scripts with special variables like \$1, \$2, and \$#.